Senior React / React Native

Interview Guide

Security · State Management · Forms · Testing · WebView Bridge · Agentic AI & MCP · Storybook, DLS & Tailwind · React 19 · Debugging

Detailed tutorial with code examples, comparison tables and model answers

October 2026

Contents

How to use this guide 4

1. Mobile Security 4

1.1 SSL Pinning . . . 4

1.2 Code Obfuscation (iXGuard, DexGuard) . . . 5

1.3 Payload Encryption (and why "SHA-256" needs care) . . . 6

2. State Management 9

2.1 RTK Query . . . 9

2.2 TanStack Query (React Query) . . . 10

2.3 RTK Query vs TanStack Query: differences and benefits . . . 11

2.4 Zustand . . . 11

3. Form Management 13

3.1 React Hook Form . . . 13

3.2 Validation with Zod . . . 13

4. Testing 15

4.1 The testing pyramid for React Native . . . 15

4.2 Unit tests with Jest . . . 15

4.3 Component tests with React Native Testing Library (RNTL) . . . 15

4.4 How to test a hook in React Native . . . 16

4.5 Maestro for E2E testing . . . 17

5. WebView and the Native Bridge 18

5.1 Two directions of communication . . . 18

5.2 React Native side . . . 18

5.3 Web side . . . 19

5.4 Security rules for WebView bridges . . . 19

6. Agentic Development and AI Tools 20

6.1 Key definitions . . . 20

6.2 Copilot CLI, Claude Code and Codex . . . 20

6.3 Agent Skills: creating and using them . . . 21

6.4 Agent Harness: what it is . . . 21

6.5 Creating agents . . . 22

6.6 MCP: Model Context Protocol . . . 23

7. Storybook, Design Language System and Tailwind **25**

7.1 Storybook . . . 25

7.2 DLS (Design Language System) . . . 25

7.3 Tailwind (and NativeWind for React Native) . . . 26

8. React 19: Features and Differences from React 18 **27**

8.1 Actions and new hooks . . . 27

8.2 Developer experience improvements . . . 27

8.3 Removed / breaking changes (React 18 to 19) . . . 28

8.4 Related: React 19.2 and React Compiler . . . 28

8.5 Summary table . . . 29

9. Debugging React Native **30**

9.1 The tools . . . 30

9.2 A systematic approach (say this in the interview) . . . 30

9.3 Common problems and fixes . . . 31

10. Rapid-Fire Practice Questions **32**

Security . . . 32

State and forms . . . 32

Testing . . . 32

Architecture and AI . . . 32

11. Final Preparation Checklist **34**

How to use this guide

This guide covers every topic on your interview prep list, in order. Each chapter has three parts:

- **Concepts**: what it is and why it exists, in plain words.
- **Code**: a small, realistic React Native / TypeScript example you can explain on a whiteboard.
- **Interview angle**: likely questions, a model answer, and traps to avoid.

> Tip: For every topic, prepare one short story from your own work in the form "situation, what I did, result". Interviewers remember stories much better than definitions.

1. Mobile Security

Banking apps face three main threats: someone reading or changing network traffic (man-in-the-middle), someone reverse-engineering the app binary, and someone tampering with data on a rooted or jailbroken device. The three items on your list answer these three threats.

1.1 SSL Pinning

The problem

Normal HTTPS trusts any certificate signed by any Certificate Authority (CA) in the device's trust store. An attacker who installs their own CA on a device (common with proxy tools like Charles, Burp or mitmproxy) can read and change every request, even over HTTPS.

The solution

SSL pinning (more correctly "certificate pinning" or "public key pinning") makes the app trust **only** specific certificates or public keys that are built into the app. If the server presents anything else, the connection fails, even if the certificate is valid for the OS.

Pin type	What is pinned	Pros	Cons
Certificate pinning	The full leaf certificate	Simple	Breaks when the certificate is renewed (often yearly)
Public key (SPKI) pinning	SHA-256 hash of the certificate's public key	Survives renewal if the key is reused	Needs care when keys rotate
Intermediate/CA pinning	The issuing CA's key	Fewer app updates	Trusts more certificates

Best practice: pin the SPKI hash, always ship at least one **backup pin** (a key you control that is not yet in use), and plan the rotation process before go-live. A wrong pin with no backup means every installed app stops working until users update.

Getting the pin

```
openssl s_client -connect api.mybank.com:443 -servername api.mybank.com </dev/null \
  | openssl x509 -pubkey -noout \
  | openssl pkey -pubin -outform der \
  | openssl dgst -sha256 -binary | openssl enc -base64
```

Option A: library (react-native-ssl-public-key-pinning)

This library uses TrustKit on iOS and OkHttp's CertificatePinner on Android, so it also covers `fetch` and Axios.

```
import { initializeSslPinning, addSslPinningErrorListener }
  from 'react-native-ssl-public-key-pinning';

await initializeSslPinning({
  'api.mybank.com': {
    includeSubdomains: true,
    publicKeyHashes: [
      'CLOmM1/OXvSPjw5UOYbAf9GKOxImEp9hhku9W90fHMk=', // current
      'hxqRlPTu1bMS/0DITB1SSu0vd4u/8l8TjPgfaAp63Gc=', // backup
    ],
  },
});

addSslPinningErrorListener((error) => {
  // report to monitoring, show a "secure connection failed" screen
});
```

Option B: Android native config (no code)

```
<!-- android/app/src/main/res/xml/network_security_config.xml -->
<network-security-config>
  <domain-config cleartextTrafficPermitted="false">
    <domain includeSubdomains="true">api.mybank.com</domain>
    <pin-set expiration="2027-12-31">
      <pin digest="SHA-256">CLOmM1/OXvSPjw5UOYbAf9GKOxImEp9hhku9W90fHMk=</pin>
      <pin digest="SHA-256">hxqRlPTu1bMS/0DITB1SSu0vd4u/8l8TjPgfaAp63Gc=</pin>
    </pin-set>
  </domain-config>
</network-security-config>
```

Reference it in AndroidManifest.xml with `android:networkSecurityConfig="@xml/network_security_config"`. On iOS the equivalent native option is `NSPinnedDomains` in Info.plist (iOS 14+) or TrustKit.

Interview angle

- **Q: Can pinning be bypassed?** Yes. On a rooted/jailbroken device, tools like Frida or Objection can hook the pinning check at runtime. That is why pinning is combined with root/jailbreak detection, runtime integrity checks (RASP), and obfuscation. Security is layers, not one feature.
- **Q: What happens when the server certificate changes?** If you pinned the public key and the key is reused, nothing. If the key changes, the backup pin must already be in the shipped app. Some teams also add a remote-config "kill switch" for pins, signed so it cannot be faked.
- **Q: Does pinning replace HTTPS?** No. It is an extra check on top of normal TLS validation.

1.2 Code Obfuscation (iXGuard, DexGuard)

Why it matters

An APK or IPA can be downloaded and decompiled. Without protection, attackers can read business logic, find API endpoints, find hard-coded keys, and patch out checks (for example, remove the root check and re-sign the app).

The tools

Tool	Platform	What it does
ProGuard / R8	Android (free, built in)	Shrinks and renames classes/methods. Basic obfuscation.
DexGuard	Android (Guardsquare, commercial)	Everything R8 does, plus string encryption, class encryption, control-flow obfuscation, asset/resource encryption, anti-tamper, anti-debug, root detection, hooking detection.
iXGuard	iOS (Guardsquare, commercial)	Same family for iOS: obfuscates Swift/Obj-C symbols and control flow, encrypts strings, adds jailbreak, debugger and tamper detection.
Hermes bytecode	React Native	JS is compiled to Hermes bytecode (.hbc) instead of plain JS text. Harder to read, but **not** real obfuscation; decompilers exist.

The React Native catch

In React Native, most business logic lives in the **JavaScript bundle**, not in native code. DexGuard and iXGuard protect native code very well, but you must also protect the JS bundle:

- Enable Hermes (default since RN 0.70), so the bundle ships as bytecode.
- Guardsquare tools can encrypt the JS bundle asset and verify its integrity at runtime.
- Optionally run a JS obfuscator (e.g. javascript-obfuscator via a Metro transformer) on sensitive modules.
- **Never** put secrets in the bundle. Obfuscation slows attackers down; it does not make a secret safe.

Enable R8 in android/app/build.gradle:

```
buildTypes {
  release {
    minifyEnabled true
    shrinkResources true
    proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-rules.pro'
  }
}
```

Interview angle

- **Q: Is obfuscation enough to store an API key in the app?** No. Anything in the app can eventually be extracted. Secrets belong on the server; the app gets short-lived tokens after authentication.
- **Q: What is RASP?** Runtime Application Self-Protection: checks running inside the app (root/jailbreak, debugger attached, hooking frameworks, app re-signed, emulator) that react by blocking, logging, or wiping sensitive data.

1.3 Payload Encryption (and why "SHA-256" needs care)

> Important correction: SHA-256 is a **hash** function, not encryption. A hash is one-way: you cannot get the original data back. If an interviewer says "payload encryption with SHA-256", politely clarify: SHA-256 is used for **integrity** (HMAC, signatures, pin hashes), while **AES** or **RSA** do the encryption. Getting this right is a strong signal.

The building blocks

Concept	Algorithm	Purpose	Reversible?
Hashing	SHA-256	Fingerprint of data; detect any change	No
HMAC	HMAC-SHA256	Hash + shared secret key: proves data was not changed and came from someone with the key	No
Symmetric encryption	AES-256-GCM	Fast encryption with one shared key; GCM also checks integrity	Yes, with the key
Asymmetric encryption	RSA-OAEP / ECDH	Public key encrypts, private key decrypts; used to share the AES key	Yes, with private key
Digital signature	RSA-PSS / ECDSA with SHA-256	Proves who sent the data	Verify only

Why encrypt the payload if we already have HTTPS?

Banks often add **application-layer encryption** (end-to-end payload encryption) on top of TLS because:

- TLS ends at the load balancer/WAF/API gateway; sensitive fields (PIN, card number, OTP) should stay encrypted until the backend service that needs them.
- It protects against logging of sensitive data in intermediate systems.
- It adds defense if pinning is bypassed on a compromised device.
- Regulators (central bank guidelines, PCI DSS) often require it for certain data.

Typical hybrid encryption flow

1. App generates a random AES-256 key and a random IV (nonce) for this request.
2. App encrypts the JSON body with AES-256-GCM.
3. App encrypts the AES key with the server's RSA public key (RSA-OAEP with SHA-256).
4. App sends: encryptedKey + iv + ciphertext + authTag. Optionally adds an HMAC-SHA256 or signature header with a timestamp and nonce to stop replay attacks.
5. Server decrypts the AES key with its private key, then decrypts the body.

```
// Conceptual example. In production use a vetted native crypto module
// (e.g. react-native-quick-crypto) - never write your own crypto.
import { randomBytes, createCipheriv, publicEncrypt, constants, createHmac }
  from 'react-native-quick-crypto';

export function encryptPayload(body: object, serverPublicKeyPem: string) {
  const aesKey = randomBytes(32);   // AES-256
  const iv = randomBytes(12);       // GCM standard nonce size
  const cipher = createCipheriv('aes-256-gcm', aesKey, iv);
  const ciphertext = Buffer.concat([
    cipher.update(JSON.stringify(body), 'utf8'),
    cipher.final(),
  ]);
  const authTag = cipher.getAuthTag();

  const encryptedKey = publicEncrypt(
    { key: serverPublicKeyPem, padding: constants.RSA_PKCS1_OAEP_PADDING, oaepHash: 'sha256' },
    aesKey,
  );
```

```
  return {
    key: encryptedKey.toString('base64'),
    iv: iv.toString('base64'),
    data: ciphertext.toString('base64'),
    tag: authTag.toString('base64'),
  };
}

// Integrity header (SHA-256 used correctly!)
export function signRequest(body: string, timestamp: string, sessionKey: Buffer) {
  return createHmac('sha256', sessionKey).update(`${timestamp}.${body}`).digest('base64');
}
```

Related security items to mention

- **Secure storage:** tokens go in iOS Keychain / Android Keystore (react-native-keychain, expo-secure-store), never AsyncStorage (plain text).
- **Biometrics:** bind biometric unlock to a Keystore/Keychain key, not a JS boolean.
- **Screen protection:** block screenshots on sensitive screens (FLAG_SECURE on Android), hide content in the app switcher.
- **No sensitive logs** in release builds; strip console.* with a Babel plugin.
- **OWASP MASVS** is the standard checklist for mobile app security; name-dropping it shows maturity.

2. State Management

First, separate two kinds of state. This single idea answers half of the state management questions:

	Server state	Client (UI) state
Examples	Accounts, transactions, user profile from the API	Theme, selected tab, modal open, wizard step, cart draft
Owner	The backend; the app holds a cached copy	The app
Problems	Caching, refetching, staleness, dedupe, retries, pagination	Sharing between components, persistence
Tools	RTK Query, TanStack Query	Zustand, Redux slices, Context, useState

2.1 RTK Query

RTK Query is the data fetching and caching layer built into Redux Toolkit. You describe your API once with `createApi`, and it generates hooks, reducers and middleware.

```
// services/bankApi.ts
import { createApi, fetchBaseQuery } from '@reduxjs/toolkit/query/react';
import type { RootState } from '../store';

export const bankApi = createApi({
  reducerPath: 'bankApi',
  baseQuery: fetchBaseQuery({
    baseUrl: 'https://api.mybank.com/v1',
    prepareHeaders: (headers, { getState }) => {
      const token = (getState() as RootState).auth.token;
      if (token) headers.set('Authorization', `Bearer ${token}`);
      return headers;
    },
  }),
  tagTypes: ['Account', 'Transaction'],
  endpoints: (builder) => ({
    getAccounts: builder.query<Account[], void>({
      query: () => '/accounts',
      providesTags: ['Account'],
    }),
    getTransactions: builder.query<Transaction[], string>({
      query: (accountId) => `/accounts/${accountId}/transactions`,
      providesTags: (result, error, accountId) => [{ type: 'Transaction', id: accountId }],
    }),
    transfer: builder.mutation<TransferResult, TransferRequest>({
      query: (body) => ({ url: '/transfers', method: 'POST', body }),
      // Refetch balances and the source account's transactions automatically
      invalidatesTags: (r, e, arg) => ['Account', { type: 'Transaction', id: arg.fromAccountId
          }],
    }),
  }),
});

export const { useGetAccountsQuery, useGetTransactionsQuery, useTransferMutation } = bankApi;
```

```
// store.ts
export const store = configureStore({
  reducer: { auth: authReducer, [bankApi.reducerPath]: bankApi.reducer },
  middleware: (getDefault) => getDefault().concat(bankApi.middleware),
});
setupListeners(store.dispatch); // enables refetchOnFocus / refetchOnReconnect
```

```
function AccountsScreen() {
  const { data, isLoading, isError, refetch } = useGetAccountsQuery();
  const [transfer, { isLoading: sending }] = useTransferMutation();
```

```
  if (isLoading) return <Spinner />;
  if (isError) return <ErrorView onRetry={refetch} />;
  return <AccountList accounts={data!} />;
}
```

Key RTK Query ideas: **tags** (providesTags / invalidatesTags) drive automatic refetching; `keepUnusedDataFor` (default 60 s) controls how long unused data stays cached; `onQueryStarted` + `updateQueryData` gives optimistic updates; `@rtk-query/codegen-openapi` can generate the whole API from an OpenAPI/Swagger file.

2.2 TanStack Query (React Query)

TanStack Query does the same job but is **not tied to Redux**. You bring any function that returns a Promise.

```
import { QueryClient, QueryClientProvider, useQuery, useMutation, useQueryClient }
  from '@tanstack/react-query';

const queryClient = new QueryClient({
  defaultOptions: { queries: { staleTime: 30_000, retry: 2 } },
});

export default function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <Navigation />
    </QueryClientProvider>
  );
}

function useAccounts() {
  return useQuery({
    queryKey: ['accounts'],
    queryFn: () => api.get<Account[]>('/accounts').then((r) => r.data),
  });
}

function useTransfer() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (req: TransferRequest) => api.post('/transfers', req),
    onSuccess: (_, req) => {
      qc.invalidateQueries({ queryKey: ['accounts'] });
      qc.invalidateQueries({ queryKey: ['transactions', req.fromAccountId] });
    },
  });
}
```

In React Native, wire focus and network status manually: use `focusManager` with AppState and `onlineManager` with @react-native-community/netinfo, because there is no browser "window focus" event.

Version 5 notes worth knowing: `cacheTime` was renamed `gcTime` (default 5 min); `isLoading` for first load became `isPending`; queries take a single object argument; `useSuspenseQuery` is first-class.

2.3 RTK Query vs TanStack Query: differences and benefits

Topic	RTK Query	TanStack Query
Dependency	Needs Redux Toolkit + store	Standalone, no Redux
API definition	Centralised in createApi; endpoints declared up front	Decentralised; any hook can call any function
Cache key	Endpoint name + serialized args (automatic)	queryKey arrays you design
Invalidation	Declarative tags (providesTags/invalidatesTags)	Imperative invalidateQueries by key prefix
Default cache life	keepUnusedDataFor = 60 s	gcTime = 5 min, staleTime = 0
DevTools	Redux DevTools: every request is an action you can time-travel	TanStack Query DevTools (dedicated cache view)
Code generation	Official OpenAPI and GraphQL codegen	Third-party (Orval, Kubb, etc.)
Infinite scroll	infiniteQuery endpoints (added in RTK 2.6)	useInfiniteQuery (mature)
Bundle	Larger if you did not already use Redux	Smaller standalone
Fits best	Apps already on Redux; large teams wanting one strict pattern	Apps without Redux; flexible, hook-first teams

When to say "RTK Query is the better choice"

- The app already uses Redux for client state, so you get one store, one DevTools, one mental model.
- You want the API layer **defined in one place**, which helps big teams and code review.
- Tag-based invalidation scales well: a "transfer" mutation invalidating "Account" refetches every screen that shows balances, with no manual key bookkeeping.
- Redux middleware/listeners can react to API actions (logging, analytics, logout on 401).
- OpenAPI codegen gives typed hooks straight from the backend contract.

Model answer

> "Both solve server-state caching, deduplication, background refetch and loading/error states. I'd pick RTK Query when the app is already on Redux Toolkit, because it plugs into the same store and DevTools, the API is defined centrally, and tag invalidation keeps related screens in sync without manual key management. I'd pick TanStack Query for apps without Redux or when I want a lighter, more flexible, hook-first setup. In both cases, server state stays out of my client-state store."

2.4 Zustand

Zustand is a tiny (around 1 KB) client-state library. A store is a hook. No provider, no reducers, no action types.

```
import { create } from 'zustand';
import { persist, createJSONStorage } from 'zustand/middleware';
import AsyncStorage from '@react-native-async-storage/async-storage';

type Prefs = {
  theme: 'light' | 'dark';
  hideBalances: boolean;
  setTheme: (t: Prefs['theme']) => void;
  toggleHideBalances: () => void;
};
```

```
export const usePrefs = create<Prefs>()(
  persist(
    (set) => ({
      theme: 'light',
      hideBalances: false,
      setTheme: (theme) => set({ theme }),
      toggleHideBalances: () => set((s) => ({ hideBalances: !s.hideBalances })),
    }),
    { name: 'prefs', storage: createJSONStorage(() => AsyncStorage) },
  ),
);

// Component: select only what you need -> re-renders only when it changes
const hide = usePrefs((s) => s.hideBalances);

// Selecting several fields: use useShallow to avoid extra re-renders
import { useShallow } from 'zustand/react/shallow';
const { theme, setTheme } = usePrefs(useShallow((s) => ({ theme: s.theme, setTheme: s.setTheme
    })));

// Outside React (e.g. in an Axios interceptor)
usePrefs.getState().setTheme('dark');
```

	Redux Toolkit	Zustand	Context
Boilerplate	Medium (slices)	Very low	Low
Provider needed	Yes	No	Yes
Re-render control	Selectors	Selectors	All consumers re-render on change
DevTools	Excellent	Via devtools middleware	None
Use outside React	store.getState()	useStore.getState()	No

Security note: never persist tokens with Zustand + AsyncStorage. Use Keychain/Keystore for secrets.

Common combination: TanStack Query (or RTK Query) for server state + Zustand for small client state. Interviewers like hearing that you know these are complementary, not competitors.

3. Form Management

3.1 React Hook Form

React Hook Form (RHF) keeps input values in refs instead of React state, so typing does not re-render the whole form. It handles values, validation, errors, touched/dirty state and submission.

In React Native there is no DOM `<input>` to `register`, so you use the **Controller** component (or `useController` hook) to connect each TextInput.

3.2 Validation with Zod

Zod defines a schema once and gives you both **runtime validation** and a **TypeScript type** (`z.infer`). The `@hookform/resolvers` package connects Zod to RHF.

```
// schemas/transfer.ts
import { z } from 'zod';

export const transferSchema = z
  .object({
    fromAccount: z.string().min(1, { message: 'Choose an account' }),
    iban: z
      .string()
      .trim()
      .regex(/^AE\d{21}$/, { message: 'Enter a valid UAE IBAN (AE + 21 digits)' }),
    amount: z.coerce
      .number({ message: 'Amount must be a number' })
      .positive({ message: 'Amount must be greater than 0' })
      .max(50_000, { message: 'Daily limit is AED 50,000' }),
    note: z.string().max(140).optional(),
    otp: z.string().length(6, { message: 'OTP must be 6 digits' }),
    confirmIban: z.string(),
  })
  .refine((d) => d.iban === d.confirmIban, {
    message: 'IBANs do not match',
    path: ['confirmIban'], // put the error on this field
  });

export type TransferForm = z.infer<typeof transferSchema>;
```

```
// TransferScreen.tsx
import { useForm, Controller } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';

export function TransferScreen() {
  const {
    control,
    handleSubmit,
    formState: { errors, isSubmitting, isValid },
  } = useForm<TransferForm>({
    resolver: zodResolver(transferSchema),
    mode: 'onBlur', // validate when leaving a field
    defaultValues: { fromAccount: '', iban: '', confirmIban: '', otp: '' },
  });

  const onSubmit = async (values: TransferForm) => {
    await transfer(values).unwrap(); // RTK Query mutation
  };

  return (
    <View>
      <Controller
        control={control}
        name="iban"
        render={({ field: { onChange, onBlur, value } }) => (
          <TextInput
            testID="iban-input"
```

```
            value={value}
            onChangeText={onChange}
            onBlur={onBlur}
            autoCapitalize="characters"
          />
        )}
      />
      {errors.iban && <Text style={styles.error}>{errors.iban.message}</Text>}
      {/* ...other fields... */}
      <Button title="Send" disabled={!isValid || isSubmitting} onPress={handleSubmit(onSubmit)}
          />
    </View>
  );
}
```

Useful RHF APIs to mention

- `watch` / `useWatch`: read a value live (useWatch re-renders only the component that uses it).
- `setError`: show server-side errors on a field (e.g. "IBAN not found").
- `reset`: reset after success or load edit data.
- `useFieldArray`: dynamic lists (multiple beneficiaries).
- `trigger`: validate one step of a multi-step wizard before moving on.
- `FormProvider` + `useFormContext`: split a big form into components.

Interview angle

- **Q: Why Zod and not just RHF rules?** One schema = types + validation; the same schema can validate API responses and be shared with a Node backend. Cross-field rules (refine/superRefine) are clean.
- **Q: parse vs safeParse?** `parse` throws a ZodError; `safeParse` returns `{ success, data | error }`. Use safeParse for API responses so a bad payload does not crash the app.
- **Q: Why is RHF fast?** Uncontrolled inputs and subscriptions: only the fields/components that subscribe to a value re-render.

4. Testing

4.1 The testing pyramid for React Native

Level	Tool	What it checks	Speed
Unit	Jest	Pure functions, reducers, Zod schemas, utils	Very fast
Hook	Jest + renderHook	Custom hook logic	Fast
Component / integration	Jest + React Native Testing Library	A screen renders and behaves correctly for the user	Fast
End-to-end	Maestro (or Detox)	Real app on simulator/device, full user journey	Slow

4.2 Unit tests with Jest

```
// utils/maskIban.ts
export const maskIban = (iban: string) => iban.slice(0, 4) + '****' + iban.slice(-4);

// utils/maskIban.test.ts
describe('maskIban', () => {
  it('keeps first 4 and last 4 characters', () => {
    expect(maskIban('AE070331234567890123456')).toBe('AE07****3456');
  });
});
```

Core Jest tools: `describe/it/expect`, matchers (`toBe`, `toEqual`, `toHaveBeenCalledWith`), mocks (`jest.fn`, `jest.mock('module')`, `jest.spyOn`), fake timers (`jest.useFakeTimers`), and coverage (`jest --coverage`). Native modules must be mocked (many libraries ship a jest mock, e.g. AsyncStorage).

4.3 Component tests with React Native Testing Library (RNTL)

RNTL's principle: **test what the user sees and does**, not implementation details. Query by text, role or label; use testID only as a fallback.

```
import { render, screen, userEvent } from '@testing-library/react-native';

jest.mock('../api', () => ({ transfer: jest.fn().mockResolvedValue({ id: 'T1' }) }));

test('shows validation error for invalid IBAN', async () => {
  const user = userEvent.setup();
  render(<TransferScreen />);

  await user.type(screen.getByTestId('iban-input'), 'XX123');
  await user.press(screen.getByRole('button', { name: 'Send' }));

  expect(await screen.findByText('Enter a valid UAE IBAN (AE + 21 digits)')).toBeOnTheScreen();
});
```

Query types: `getBy*` (must exist, throws), `queryBy*` (may not exist, returns null; use for "not shown" checks), `findBy*` (async, waits). Use `waitFor` for async assertions. Prefer `userEvent` over `fireEvent` because it simulates the full event sequence a real user would trigger.

4.4 How to test a hook in React Native

Use **renderHook** from @testing-library/react-native. It renders a tiny test component that calls your hook and gives you `result.current`. Wrap state changes in `act`.

A simple hook

```
// useCounter.ts
export function useCounter(initial = 0) {
  const [count, setCount] = useState(initial);
  const increment = () => setCount((c) => c + 1);
  return { count, increment };
}

// useCounter.test.ts
import { renderHook, act } from '@testing-library/react-native';

test('increments', () => {
  const { result } = renderHook(() => useCounter(5));
  act(() => result.current.increment());
  expect(result.current.count).toBe(6);
});

test('responds to new props', () => {
  const { result, rerender } = renderHook(({ start }) => useCounter(start), {
    initialProps: { start: 1 },
  });
  expect(result.current.count).toBe(1);
  rerender({ start: 10 }); // note: useState ignores new initial values
  expect(result.current.count).toBe(1);
});
```

A hook that needs a provider (TanStack Query / Redux)

```
const createWrapper = () => {
  const qc = new QueryClient({ defaultOptions: { queries: { retry: false } } });
  return ({ children }: { children: React.ReactNode }) => (
    <QueryClientProvider client={qc}>{children}</QueryClientProvider>
  );
};

test('useAccounts returns data', async () => {
  jest.spyOn(api, 'get').mockResolvedValue({ data: [{ id: '1', balance: 100 }] });
  const { result } = renderHook(() => useAccounts(), { wrapper: createWrapper() });

  await waitFor(() => expect(result.current.isSuccess).toBe(true));
  expect(result.current.data).toHaveLength(1);
});
```

For RTK Query hooks, the wrapper is `<Provider store={setupStore()}>` with a fresh store per test. For network mocking, **MSW (Mock Service Worker)** intercepts requests so you test the real fetch code.

Interview checklist for hook tests

- Fresh provider/store per test, so tests do not share cache.
- Turn off retries in tests, or error tests become slow.
- Use fake timers for debounce/polling hooks: `jest.useFakeTimers()` then `act(() => jest.advanceTimersByTime(500))`.
- Test the hook's **contract** (inputs, returned values, effects), not its internal state.

4.5 Maestro for E2E testing

Maestro is a mobile UI testing framework where tests ("flows") are **YAML files**. It drives the real app on a simulator/emulator or device, waits automatically for elements (fewer flaky tests), and needs no changes to the app code.

```
# .maestro/transfer.yaml
appId: com.mybank.app
---
- launchApp:
    clearState: true
- tapOn: "Log in"
- tapOn:
    id: "username-input"     # matches testID in React Native
- inputText: "demo_user"
- tapOn:
    id: "password-input"
- inputText: ${PASSWORD}     # injected: maestro test -e PASSWORD=... flow.yaml
- tapOn: "Continue"
- assertVisible: "Accounts"
- tapOn: "Transfer"
- runFlow: subflows/fill-transfer.yaml   # reusable steps
- tapOn: "Send"
- assertVisible: "Transfer successful"
- takeScreenshot: transfer-success
```

```
curl -Ls "https://get.maestro.mobile.dev" | bash   # install
maestro test .maestro/transfer.yaml                # run one flow
maestro test .maestro/                              # run all flows
maestro studio                                      # inspect elements, record steps
```

	Maestro	Detox
Language	YAML	JavaScript/TypeScript (Jest)
Synchronisation	Black-box; automatic waits and retries	Grey-box; knows when RN is idle
Setup	Very easy, no app changes	Needs native build config
Works with	Any app (RN, Flutter, native, WebViews)	React Native mainly
CI	Maestro Cloud or any CI with emulators	Any CI with emulators

Good practice: add a `testID` to important elements, keep test accounts and mocked OTP in a test environment, run smoke flows on every PR and the full suite nightly.

5. WebView and the Native Bridge

Banks often reuse web content inside the app (KYC journeys, card offers, third-party payment pages). `react-native-webview` renders it, and a **bridge** lets the web page and the React Native app talk to each other.

5.1 Two directions of communication

Direction	Mechanism
Web to Native	Page calls `window.ReactNativeWebView.postMessage(string)`; RN receives it in `onMessage`
Native to Web	RN calls `webviewRef.current.injectJavaScript('...')`, or sets `injectedJavaScript` / `injectedJavaScriptBeforeContentLoaded`

Messages are **strings only**, so use JSON with a clear contract: `{ id, type, payload }`. The `id` lets you match a response to a request (like a mini RPC).

5.2 React Native side

```
import { WebView, WebViewMessageEvent } from 'react-native-webview';

const ALLOWED_ORIGIN = 'https://kyc.mybank.com';

export function KycWebView() {
  const ref = useRef<WebView>(null);

  const reply = (id: string, payload: unknown) => {
    const msg = JSON.stringify({ id, payload });
    ref.current?.injectJavaScript(`window.onNativeMessage(${msg}); true;`);
  };

  const onMessage = async (e: WebViewMessageEvent) => {
    if (!e.nativeEvent.url.startsWith(ALLOWED_ORIGIN)) return;  // trust check
    let msg: { id: string; type: string; payload?: any };
    try { msg = JSON.parse(e.nativeEvent.data); } catch { return; }

    switch (msg.type) {
      case 'GET_USER_TOKEN':
        reply(msg.id, { token: await getShortLivedToken() });
        break;
      case 'OPEN_CAMERA':
        reply(msg.id, await captureDocument());
        break;
      case 'CLOSE':
        navigation.goBack();
        break;
    }
  };

  return (
    <WebView
      ref={ref}
      source={{ uri: `${ALLOWED_ORIGIN}/start` }}
      originWhitelist={[ALLOWED_ORIGIN]}
      onMessage={onMessage}
      onShouldStartLoadWithRequest={(req) => req.url.startsWith(ALLOWED_ORIGIN)}
      javaScriptEnabled
      allowFileAccess={false}
      injectedJavaScriptBeforeContentLoaded={`window.isNativeApp = true; true;`}
    />
  );
}
```

Note the `true;` at the end of injected scripts: it avoids a silent failure on some platforms.

5.3 Web side

```
const pending = new Map();

function callNative(type, payload) {
  return new Promise((resolve) => {
    const id = crypto.randomUUID();
    pending.set(id, resolve);
    window.ReactNativeWebView.postMessage(JSON.stringify({ id, type, payload }));
  });
}

window.onNativeMessage = ({ id, payload }) => {
  pending.get(id)?.(payload);
  pending.delete(id);
};

// usage
const { token } = await callNative('GET_USER_TOKEN');
```

5.4 Security rules for WebView bridges

- Check the origin/URL of every message; restrict navigation with `onShouldStartLoadWithRequest`.
- Expose a small, explicit list of actions. Never `eval` received strings.
- Pass short-lived, scoped tokens, never long-lived refresh tokens.
- Disable file access; avoid `mixedContentMode="always"`.
- Apply SSL pinning to WebView traffic too (WebViews do not always use the same network stack as fetch).

Interview angle

- **Q: Difference between this "bridge" and the React Native bridge?** The WebView bridge is message passing between a web page and RN. The (old) React Native bridge was the async JSON channel between JS and native modules. The **New Architecture** replaced it with **JSI** (direct C++ calls), **TurboModules** (lazy-loaded, typed native modules) and **Fabric** (new renderer). New Architecture is the default since RN 0.76.

6. Agentic Development and AI Tools

6.1 Key definitions

Term	Simple definition
LLM	The model: takes text in, produces text out. On its own it cannot act.
Agent	An LLM running in a **loop** with **tools**: it plans, calls a tool (read file, run tests, call API), looks at the result, and decides the next step until the goal is done.
Agentic development	Developing software with agents doing multi-step work (read code, edit, run tests, fix) while the developer sets goals, reviews and approves.
Agentic workflow	A designed process where one or more agents (and humans) complete a task in steps, e.g. ticket to plan to code to tests to PR to review.
Agent harness	The software **around** the model that turns it into an agent (see 6.4).
Tool	A function the model may call, described by a name, description and JSON schema.
MCP	A standard protocol to plug tools and data sources into any agent (see 6.6).
Skill	A packaged, reusable set of instructions (plus optional scripts) an agent loads when relevant (see 6.3).
Subagent	A helper agent with its own context and tool set that the main agent delegates to.

Workflows vs agents

- **Workflow**: the steps are fixed in code (step 1, step 2, step 3), and the LLM fills each step. Predictable, easy to test.
- **Agent**: the LLM decides the steps dynamically. Flexible, good for open-ended tasks, less predictable.
- Common workflow patterns: **prompt chaining** (output of one step feeds the next), **routing** (classify then send to a specialised prompt), **parallelisation** (several reviewers at once), **orchestrator-workers** (one agent splits work across subagents), **evaluator-optimiser** (one generates, another critiques, loop).

6.2 Copilot CLI, Claude Code and Codex

All three are **terminal/IDE coding agents**: they read your repository, edit files, run commands and tests, and work through tasks in a loop with your approval.

	GitHub Copilot (CLI / agent mode)	Claude Code	OpenAI Codex
Vendor	GitHub / Microsoft	Anthropic	OpenAI
Where	Terminal (copilot), VS Code, GitHub.com coding agent	Terminal, IDE extensions, desktop, web	Terminal (codex CLI), IDE, cloud
Project instructions	.github/copilot-instructions.md, AGENTS.md	CLAUDE.md	AGENTS.md
Extensibility	MCP servers, custom agents	MCP, skills, hooks, subagents, plugins, slash commands	MCP, skills, config
Strength	Tight GitHub integration (issues, PRs)	Deep agentic coding, rich customisation	OpenAI models, cloud task runs

How to use them well (good interview points)

- Give context: a project instruction file with build/test commands, architecture and conventions.
- **Plan first** for big changes (plan mode), review the plan, then implement.
- Keep a human in the loop: review diffs, run tests, never auto-merge security-sensitive code.
- Give the agent a way to verify its work: tests, linters, type checks, a running simulator.
- Be careful with permissions: allow-list safe commands; do not give agents production secrets.
- Bank context: check company policy about which code and data can go to which AI providers.

6.3 Agent Skills: creating and using them

A **skill** is a folder containing a `SKILL.md` file with YAML frontmatter (name, description) and instructions, plus optional scripts, templates and reference files. The agent sees only the **name and description** at first; when a task matches, it loads the full instructions (this is called **progressive disclosure**, which keeps the context window small). Skills started in Claude Code and are now an open format also supported by other agents.

```
.claude/skills/rn-component/
  SKILL.md
  templates/Component.tsx
  templates/Component.test.tsx
  templates/Component.stories.tsx
```

```
---
name: rn-component
description: Create a new React Native component following our DLS rules.
  Use when the user asks for a new screen element or UI component.
---

# Creating a DLS component

1. Read the design in Figma using the Figma MCP (get_design_context).
2. Use only tokens from @mybank/dls-tokens. Never hard-code colors or spacing.
3. Copy the three files in templates/ and rename them.
4. Add a Storybook story covering every variant and the disabled state.
5. Add RNTL tests: renders, press handler, accessibility label.
6. Run `yarn lint && yarn test <ComponentName>` and fix all failures.
```

Example skills you could build and talk about: "create-api-endpoint" (RTK Query endpoint + types + tests), "write-maestro-flow", "release-notes" from merged PRs, "security-checklist" before merging (no secrets, no console logs, Keychain for tokens).

Skill vs MCP vs project instructions: project instructions (CLAUDE.md/AGENTS.md) are always loaded; skills are loaded on demand and teach **how** to do a task; MCP servers give **access** to external systems (tools and data).

6.4 Agent Harness: what it is

The **harness** is everything around the model that makes it a working agent. The model provides intelligence; the harness provides the body.

- **The agent loop:** send context to the model, receive a tool call, execute it, append the result, repeat until done.
- **Tools:** file read/edit, shell, search, web fetch, MCP tools.
- **Context management:** system prompt, project instructions, loading skills, summarising/compacting long conversations.

- **Permissions and safety:** which tools need approval, sandboxing, allow/deny lists.
- **Hooks:** your scripts that run on events (e.g. run the formatter after every edit, block edits to .env files).
- **Subagents and orchestration:** spawning helper agents with separate contexts.
- **Memory and state:** persistent notes across sessions.
- **UI / interface:** terminal, IDE, web, or headless SDK mode for CI.

> One-line answer: "Claude Code, Copilot CLI and Codex are agent harnesses. The same model can perform very differently depending on the harness: the tools, context strategy and guardrails around it."

6.5 Creating agents

Option 1: subagents inside Claude Code (no code)

```
<!-- .claude/agents/rn-reviewer.md -->
---
name: rn-reviewer
description: Reviews React Native changes for performance, accessibility and security.
tools: Read, Grep, Glob, Bash
---
You are a senior React Native reviewer. For the current diff check:
- unnecessary re-renders (missing memo, inline objects in FlatList)
- accessibility labels and roles
- secrets, tokens in AsyncStorage, console logs
Report findings ordered by severity with file:line references.
```

Option 2: build an agent with an SDK

The Claude Agent SDK gives you the same harness as Claude Code as a library (TypeScript and Python). Other options include the OpenAI Agents SDK, LangGraph and Microsoft frameworks.

```
import { query } from '@anthropic-ai/claude-agent-sdk';

for await (const message of query({
  prompt: 'Find all screens that store tokens in AsyncStorage and propose a fix.',
  options: {
    allowedTools: ['Read', 'Grep', 'Glob'],   // read-only agent
    mcpServers: {
      github: { type: 'http', url: 'https://api.githubcopilot.com/mcp/' },
    },
  },
})) {
  if (message.type === 'result') console.log(message.result);
}
```

Option 3: the raw loop (shows you understand the concept)

```
const tools = [{ name: 'run_tests', description: 'Run Jest', input_schema: {...} }];
let messages = [{ role: 'user', content: task }];

while (true) {
  const res = await llm.create({ model, tools, messages });
  messages.push({ role: 'assistant', content: res.content });
  if (res.stop_reason !== 'tool_use') break;          // finished
  const results = await runToolCalls(res.content);    // execute requested tools
  messages.push({ role: 'user', content: results });  // feed results back
}
```

Agent design tips: clear tool descriptions, small focused toolsets, least-privilege permissions, a stopping condition and step limit, logging/tracing every step, and **evals** (test cases that measure if the agent does the

task correctly).

6.6 MCP: Model Context Protocol

MCP is an open protocol (introduced by Anthropic in late 2024, now widely adopted) that standardises how AI apps connect to tools and data. Think of it as "USB-C for AI": write a server once, use it in Claude Code, Copilot, Cursor, Codex and others.

Part	Role
Host	The AI app (Claude Code, VS Code Copilot)
Client	Connection inside the host, one per server
Server	Exposes capabilities of a system (GitHub, Figma, Azure DevOps)
Tools	Actions the model can call (create_issue, get_work_item)
Resources	Data the app can read (files, records)
Prompts	Reusable prompt templates
Transports	stdio (local process) or Streamable HTTP (remote, with OAuth)

MCPs to talk about from your work (prepare your own examples)

Your notes say to mention the MCPs you used at RAKBANK. Prepare a concrete story for each one, using this structure:

- **Figma MCP:** pulls design context (layout, components, variables/tokens, screenshots) from a selected Figma frame so the agent can generate code that matches the design and maps to existing DLS components. Story idea: "I selected the frame, the agent read the tokens and generated the component using our DLS components; I reviewed spacing and accessibility, and it cut first-draft time significantly."
- **Azure DevOps (ADO) MCP:** reads and updates work items, boards, repos, pull requests, pipelines and wikis. Story idea: "The agent read the user story and acceptance criteria from ADO, created the implementation plan, and later linked the PR and updated the work item status."
- **GitHub MCP:** issues, PRs, code search, reviews, Actions runs. Story idea: "Reviewing PR comments, summarising CI failures, creating issues for follow-ups."

```
# Adding MCP servers in Claude Code (examples)
claude mcp add --transport http github https://api.githubcopilot.com/mcp/
claude mcp add --transport http figma https://mcp.figma.com/mcp
claude mcp list
```

Project-level config can live in `.mcp.json` so the whole team shares it. VS Code uses `.vscode/mcp.json`.

Writing a tiny MCP server (TypeScript)

```
import { McpServer } from '@modelcontextprotocol/sdk/server/mcp.js';
import { StdioServerTransport } from '@modelcontextprotocol/sdk/server/stdio.js';
import { z } from 'zod';

const server = new McpServer({ name: 'dls-tokens', version: '1.0.0' });

server.tool(
  'get_token',
  'Look up a design token value by name, e.g. color.primary.500',
  { name: z.string() },
  async ({ name }) => ({
    content: [{ type: 'text', text: JSON.stringify(tokens[name] ?? 'not found') }],
  }),
);

await server.connect(new StdioServerTransport());
```

MCP security points (important for a bank)

- Only use trusted/approved servers; a malicious server can inject instructions (**prompt injection**) through tool results.
- Least privilege: read-only tokens where possible, scoped PATs, no production data.
- Review what each tool can do before allowing it without approval.

7. Storybook, Design Language System and Tailwind

7.1 Storybook

Storybook is a workshop for building and documenting UI components **in isolation**: each component state is a "story". For React Native there is `@storybook/react-native` (runs on device), and many teams also run the same stories on the web with react-native-web for documentation and visual tests.

```
// Button.stories.tsx  (Component Story Format)
import type { Meta, StoryObj } from '@storybook/react-native';
import { Button } from './Button';

const meta = {
  title: 'DLS/Button',
  component: Button,
  args: { label: 'Continue', onPress: () => {} },
  argTypes: { variant: { control: 'select', options: ['primary', 'secondary', 'ghost'] } },
} satisfies Meta<typeof Button>;
export default meta;

type Story = StoryObj<typeof meta>;
export const Primary: Story = { args: { variant: 'primary' } };
export const Disabled: Story = { args: { variant: 'primary', disabled: true } };
export const Loading: Story = { args: { variant: 'primary', loading: true } };
```

Why teams use it: develop without navigating the app, review UI with designers and QA, living documentation, controls to test every prop, and the base for visual regression tests (e.g. Chromatic on the web build) and interaction/a11y tests.

7.2 DLS (Design Language System)

A DLS is the single source of truth for how the product looks and behaves. Your notes mention RAKBANK has a DLS for both React and React Native: be ready to explain how it is structured and how you contributed.

Layer	Contents
Design tokens	Colors, typography, spacing, radius, shadows, motion; light/dark themes
Primitives	Box, Text, Stack, Icon
Components	Button, Input, Card, BottomSheet, OTPInput, AmountInput
Patterns	Forms, empty states, error pages, transaction lists
Guidelines	Accessibility, content/tone, do and don't

How it is usually delivered:

- Tokens defined once (Figma variables / Tokens Studio JSON) and transformed with **Style Dictionary** into TS for React Native and CSS variables for web.
- Shared logic and types; separate platform implementations for React and React Native where needed.
- Published as versioned npm packages (semantic versioning, changelog, Changesets), often in a monorepo.
- Storybook as the documentation site; Figma components mapped to code (Figma Code Connect), which also helps the Figma MCP produce correct code.

Talking points: consistency across web and mobile, faster delivery, accessibility built in once, theming (e.g. different segments/brands), and governance (how new components are proposed and reviewed).

7.3 Tailwind (and NativeWind for React Native)

Tailwind CSS is a **utility-first** CSS framework: you style with small classes (`p-4`, `text-lg`, `bg-blue-600`) instead of writing custom CSS. React Native has no CSS, so **NativeWind** compiles Tailwind classes into React Native styles.

```
// Web (React + Tailwind)
<button className="rounded-lg bg-primary-600 px-4 py-2 text-white hover:bg-primary-700
    disabled:opacity-50">
  Continue
</button>

// React Native (NativeWind)
<Pressable className="rounded-lg bg-primary-600 px-4 py-3 active:opacity-80">
  <Text className="text-center font-semibold text-white">Continue</Text>
</Pressable>
```

```
// tailwind.config.js: connect Tailwind to DLS tokens
const tokens = require('@mybank/dls-tokens');
module.exports = {
  content: ['./src/**/*.{ts,tsx}'],
  presets: [require('nativewind/preset')],
  theme: { extend: { colors: tokens.colors, spacing: tokens.spacing } },
};
```

Pros: fast styling, consistent spacing/color scale, no naming problems, small CSS output (unused classes removed), dark mode with `dark:`. Cons: long class strings (use `clsx`/`tailwind-merge`/`cva` for variants), and a learning curve. Key point for a DLS: **map Tailwind's theme to the design tokens**, so utilities can only use approved values.

8. React 19: Features and Differences from React 18

React 19 became stable in December 2024 (with 19.1 and 19.2 in 2025). React Native adopted it in **RN 0.78**.

8.1 Actions and new hooks

Actions are async functions used in transitions. React manages pending state, errors, optimistic updates and form reset for you.

```
// React 18: manual pending/error state
const [isPending, setIsPending] = useState(false);
const [error, setError] = useState(null);
const submit = async () => {
  setIsPending(true);
  const err = await updateName(name);
  setIsPending(false);
  if (err) setError(err);
};

// React 19: useActionState
const [error, submitAction, isPending] = useActionState(
  async (prevState, formData) => {
    const err = await updateName(formData.get('name'));
    if (err) return err;
    redirect('/profile');
    return null;
  },
  null,
);
// <form action={submitAction}> ... <button disabled={isPending}>Save</button>
```

API	What it does
useActionState	Returns [state, action, isPending] for an async action
useFormStatus (react-dom)	Child component reads parent form's pending state; no prop drilling
useOptimistic	Show the expected result immediately, roll back automatically on failure
use()	Read a Promise (suspends) or Context; can be called conditionally, unlike other hooks
startTransition(async fn)	Transitions now accept async functions
form action={fn}	Pass a function to form action; form resets on success (web)

```
// useOptimistic: show the message right away
const [optimisticMsgs, addOptimistic] = useOptimistic(
  messages,
  (state, newMsg) => [...state, { text: newMsg, sending: true }],
);

// use(): read context conditionally / unwrap a promise inside Suspense
function Balance({ balancePromise }) {
  const balance = use(balancePromise); // suspends until resolved
  return <Text>{balance}</Text>;
}
```

8.2 Developer experience improvements

- **ref as a prop:** function components receive `ref` directly. `forwardRef` is no longer needed (will be deprecated).

- **Context as provider:** `<ThemeContext value="dark">` instead of `<ThemeContext.Provider value="dark">`.
- **Ref cleanup functions:** a ref callback can return a cleanup function.
- **useDeferredValue initial value:** `useDeferredValue(value, initialValue)`.
- **Better error reporting:** one clear error instead of duplicates; new root options `onCaughtError`, `onUncaughtError`; much better hydration mismatch diffs.
- **Document metadata (web):** render `<title>`, `<meta>`, `<link>` anywhere in the tree; React hoists them to head.
- **Stylesheets and async scripts (web):** `<link rel="stylesheet" precedence="...">` handled with Suspense; resource preloading APIs `preload`, `preinit`, `prefetchDNS`, `preconnect`.
- **Server Components and Server Actions** are stable (used via frameworks like Next.js; `"use server"` / `"use client"` directives).
- **Custom Elements** are fully supported.

```
// React 18
const Input = forwardRef((props, ref) => <TextInput ref={ref} {...props} />);

// React 19
function Input({ ref, ...props }) {
  return <TextInput ref={ref} {...props} />;
}
```

8.3 Removed / breaking changes (React 18 to 19)

- `propTypes` checks removed (use TypeScript); `defaultProps` removed for function components (use default parameters).
- Legacy context (`contextTypes`), string refs, `createFactory` removed.
- `ReactDOM.render` / `hydrate` / `unmountComponentAtNode` removed (use `createRoot` / `hydrateRoot`).
- `react-test-renderer` deprecated; `act` moved from `react-dom/test-utils` to `react`.
- Errors in render are no longer re-thrown; they go to `window.reportError` / `console.error`.
- TypeScript: `useRef` requires an argument; ref callbacks must not implicitly return values.
- Codemods help migrate: `npx codemod@latest react/19/migration-recipe`.

8.4 Related: React 19.2 and React Compiler

- **React 19.2:** `<Activity>` (hide parts of the UI while keeping their state, like a background tab) and `useEffectEvent` (read latest props/state in an effect without re-running it).
- **React Compiler** (stable 1.0 in late 2025): a build-time tool that adds memoisation automatically, so most `useMemo`, `useCallback` and `React.memo` become unnecessary. It works with React 17+ but is promoted together with 19. Works with React Native via Babel plugin.

8.5 Summary table

Area	React 18	React 19
Async updates	Manual loading/error state	Actions, useActionState, useOptimistic
Forms	onSubmit handlers	form action, useFormStatus
Refs	forwardRef	ref is a normal prop; ref cleanup
Context	Context.Provider	Context as provider; use(Context)
Data reading	useEffect + state, or library	use(promise) with Suspense
Metadata	react-helmet etc.	Built-in title/meta/link
Server	Experimental RSC	Server Components/Actions stable
Memoisation	Manual	React Compiler (optional)
Big 18 features (know them too)	createRoot, concurrent rendering, automatic batching, useTransition, useDeferredValue, useId, Suspense on server	

9. Debugging React Native

9.1 The tools

Tool	Use it for
React Native DevTools	The official debugger since RN 0.76 (replaced Flipper and remote Chrome debugging). Press **j** in Metro or open from the Dev Menu. Console, breakpoints, sources, React Components tree, Profiler; newer versions add Network and Performance panels. Works with Hermes.
Dev Menu	Cmd+D (iOS sim), Cmd+M / Ctrl+M (Android emulator) or shake device: reload, Fast Refresh, Perf Monitor, Element Inspector.
LogBox	In-app warnings/errors with stack traces. `LogBox.ignoreLogs([...])` for known noise.
Reactotron	Inspect API calls, Redux/Zustand state, async storage, timelines.
Redux DevTools	Actions and state history, including RTK Query requests.
Xcode	Native iOS crashes, Instruments (memory leaks, CPU, Time Profiler), device logs (Console.app).
Android Studio	Logcat (`adb logcat`), Profiler (CPU, memory), Layout Inspector.
Proxy tools	Charles / Proxyman to inspect traffic (debug builds only; pinning blocks this in release).
Crash reporting	Sentry, Firebase Crashlytics with source maps for readable JS stacks.
why-did-you-render / React Profiler	Find unnecessary re-renders.

9.2 A systematic approach (say this in the interview)

1. **Reproduce** reliably: device, OS version, build type (debug vs release), steps.
2. **Locate the layer:** JS (red screen, JS stack), native (crash with no JS stack, check Xcode/Logcat), network (check proxy/DevTools Network), or build (Metro/Gradle/CocoaPods errors).
3. **Inspect:** breakpoints in RN DevTools, logs, state in Redux DevTools/Reactotron.
4. **Fix and prevent:** add a test that reproduces the bug, then fix.

9.3 Common problems and fixes

Problem	Typical cause / fix
Works in debug, crashes in release	Missing ProGuard/R8 keep rules, env variables missing, Hermes differences. Test release builds early; use `adb logcat` / Xcode device logs.
Red screen "Unable to resolve module"	Metro cache: `npx react-native start --reset-cache`; check imports and package install.
iOS build errors after upgrade	`cd ios && pod install`, clear DerivedData; check the RN Upgrade Helper.
Android build cache issues	`cd android && ./gradlew clean`.
Slow FlatList	Use `keyExtractor`, `getItemLayout`, memoised `renderItem`, avoid inline objects; consider FlashList.
Janky animations	Run animations on the UI thread with Reanimated or `useNativeDriver: true`.
Memory leaks	Clean up listeners/timers in effect cleanups; profile with Instruments / Android Profiler.
Production crash with unreadable stack	Upload source maps to Sentry/Crashlytics.
Too many re-renders	React DevTools Profiler ("highlight updates"), selectors in Zustand/Redux, memoisation or React Compiler.

```
npx react-native doctor             # check environment setup
npx react-native log-android        # JS + native logs
adb reverse tcp:8081 tcp:8081        # device cannot reach Metro
npx react-native run-android --mode release
```

10. Rapid-Fire Practice Questions

Practise answering each one aloud in under one minute.

Security

- **What is SSL pinning and why is it needed if we have HTTPS?** HTTPS trusts any CA on the device; pinning trusts only our key, blocking MITM through installed or rogue CAs.
- **How do you rotate a pinned certificate safely?** Ship the backup pin in an earlier release; switch the server to the backup key; then add a new backup in the next release.
- **Is SHA-256 encryption?** No, it is a one-way hash. Use AES-GCM for encryption, RSA/ECDH for key exchange, HMAC-SHA256 or signatures for integrity.
- **Where do you store tokens?** Keychain / Keystore via react-native-keychain; never AsyncStorage.
- **What do DexGuard and iXGuard add over R8?** String/class encryption, control-flow obfuscation, anti-tamper, anti-debug, root/jailbreak and hook detection.

State and forms

- **Server vs client state?** Server state is a cached copy of backend data (RTK Query/TanStack Query); client state is owned by the app (Zustand/Redux/useState).
- **How does RTK Query invalidate cache?** Queries `providesTags`, mutations `invalidatesTags`; matching queries refetch automatically.
- **staleTime vs gcTime in TanStack Query?** staleTime: how long data is considered fresh (no refetch). gcTime: how long unused data stays in memory before removal.
- **Why Zustand over Context?** No provider, selectors avoid re-rendering every consumer, usable outside React.
- **Why Controller in React Native forms?** RN TextInput is not a DOM input, so RHF cannot `register` it directly.
- **How do you validate that two fields match in Zod?** `.refine()` (or `.superRefine()`) on the object, with `path` pointing to the field.

Testing

- **How do you test a custom hook?** `renderHook` from RNTL, `act` for updates, `wrapper` for providers, `waitFor` for async.
- **getBy vs queryBy vs findBy?** Throws if missing / returns null / async waits.
- **Why Maestro?** Simple YAML, automatic waiting reduces flakiness, black-box so it works for any app, easy CI.

Architecture and AI

- **What is the New Architecture?** JSI + TurboModules + Fabric; replaces the async JSON bridge with direct, synchronous-capable C++ calls.
- **How does a WebView talk to RN?** postMessage + onMessage one way, injectJavaScript the other; JSON messages with IDs; validate origin.

- **What is an agent harness?** The loop, tools, context management, permissions, hooks and UI around the model.
- **What is MCP?** An open protocol connecting AI hosts to tools/data servers via tools, resources and prompts over stdio or HTTP.
- **Skill vs MCP?** A skill teaches the agent how to do a task (instructions + scripts, loaded on demand); MCP gives access to an external system.
- **What changed from React 18 to 19?** Actions and useActionState/useOptimistic/useFormStatus, use(), ref as a prop, Context as provider, metadata support, stable Server Components, removed propTypes/defaultProps for functions/string refs/ReactDOM.render.

11. Final Preparation Checklist

- Prepare 2-minute stories: a security feature you built, a performance fix, a hard bug you debugged, a DLS component you created, and how you used AI agents and MCPs (Figma, ADO, GitHub) day to day.
- Build a tiny demo repo: RTK Query + Zustand + RHF/Zod form + one hook test + one Maestro flow. Being able to say "I tried this last week" is powerful.
- Create one real Agent Skill and one subagent, so you can describe them from experience.
- Read the React 19 upgrade guide once, and know which RN version your apps run.
- Be honest about what you have not used, and explain how you would learn it quickly. That is better than guessing.
- Ask the interviewers good questions: how they use AI agents in their workflow, how their DLS is governed, how they test (unit vs E2E balance), and their release process.

> Good luck. Clear, structured answers with real examples beat memorised definitions every time.